Phase 1: Systematic Problem Identification in ICL

In-Context Learning as Latent Space Manipulation

Author:
Pratyay Dutta
Department of Computer Science, University of California, Riverside
Lawrence Livermore National Laboratory

Advisor:
Prof. Bir Bhanu

Status:
Phase 1: Problem Identification & Diagnostic Experimentation

Date:
June 2026

Phase 1: Systematic Problem Identification in ICL as Latent Sp

Author: Pratyay Dutta, UC Riverside **Date**: June 2026 **Advisor**: Prof. Bir Bhanu **Status**: Phase 1 -- Problem Identification & Diagnostic Experimentation

Executive Summary

This plan establishes a rigorous Phase 1 research protocol for identifying specific, publishable problems in the In-Context Learning (ICL) landscape, viewed through the lens of **latent space manipulation**. Building on the theoretical synthesis connecting ICL attention dynamics to classical Query Vector Movement (QVM), we now integrate **Feature Relevance Estimation (FRE)** and design a systematic battery of diagnostic experiments to expose failure modes, quantify vulnerabilities, and converge on a precise problem formulation.

The core thesis: ICL can be formally modeled as a latent space manipulation procedure comprising three components from classical CBIR: QVM (query vector movement via attention), FRE (feature axis weighting via learned attention patterns), and BI (Bayesian inference via posterior task estimation). By exposing where this formalism breaks down, we identify novel research opportunities.

Background & Theoretical Foundation

What We Know (From Your Research Log)

Your research establishes a mathematical bridge between:

Classical CBIR (TPAMI 2005)	Modern ICL (Transformers)				
q_new = alpha q_old + beta frac1	D^+	sum_x in D^+ x - gamma frac1	D^-	sum_x in D^- x	h_q^(new) = h_q^(old) + sum_i in C softmax(frac(W_q h_q^(old))(W_k h_i)^Tsqrtd) W_v h_i
Residual = alpha q_old	Residual = h_q^(old)				
Static relevance weight beta frac1	D^+		Dynamic attention score softmax(*)		
Missing: -gamma repulsion term	**No negative demonstrations**				

What We Now Add: Feature Relevance Estimation (FRE)

From the TPAMI 2005 paper (Yin, Bhanu, Chang, Dong), FRE introduces **per-axis weighting** of the feature space. In classical CBIR:

$$w_j^{(t+1)} = w_j^{(t)} \cdot \frac{\sigma_j^{(\text{irrelevant})}}{\sigma_j^{(\text{relevant})} + \epsilon}$$

Where w_j is the weight for feature dimension j, and σ_j measures the spread of relevant vs. irrelevant examples along that dimension. Dimensions where relevant examples cluster tightly (low $\sigma_j^{(\text{relevant})}$) receive higher weights.

The ICL Analogy: In transformers, different latent dimensions (axes) of the hidden representation encode different features. During ICL, the model implicitly learns which dimensions are "relevant" for the current task. This is analogous to:

1. **Multi-head attention** -> Different heads attend to different feature subspaces (implicit FRE)
2. **Layer normalization** -> Rescales feature axes (implicit axis weighting)
3. **Sparse autoencoders** -> Decompose activations into interpretable feature axes (explicit FRE)

> **IMPORTANT**
> ***Key Research Question**: Does ICL perform implicit Feature Relevance Estimation? If so, how well does it do it, and where does it fail? This is the bridge between your QVM work and a concrete, testable hypothesis.*

The Unified ICL-CBIR Framework

We propose that ICL performs three operations in latent space, mirroring the three RF techniques from TPAMI 2005:

```
+-----------------------------------------------------------+
|              ICL as Latent Space Manipulation             |
+---------------------+-------------------------------------+
|   CBIR Component    |   Transformer Analogue              |
+---------------------+-------------------------------------+
|   QVM               |   Attention-weighted residual       |
|   (Query Movement)  |   update of query representation    |
+---------------------+-------------------------------------+
|   FRE               |   Implicit per-dimension feature    |
|   (Feature Weights) |   weighting via attention heads     |
+---------------------+-------------------------------------+
|   BI                |   Posterior task inference:         |
|   (Bayesian Inf.)   |   P(y|x,C) approx. integral P(y|x,?)P(?|C)d?|
+---------------------+-------------------------------------+
|   ? Term            |   ??? (MISSING -- Research Gap)     |
|   (Neg. Feedback)   |                                     |
+---------------------+-------------------------------------+
```

Open Questions for User Review

> **IMPORTANT**
> ***Q1: Model Selection** -- Which LLMs do you have access to for experimentation? The plan is designed for models where you can access internal activations (e.g., GPT-2, LLaMA-2/3, Pythia, OLMo). Open-weight models are essential for the probing experiments. Do you have GPU access for running these?*

> **IMPORTANT**
> ***Q2: Scope of Tasks** -- The experiments below span NLP classification, reasoning, and potentially vision-language tasks. Should we focus exclusively on text classification first (simpler, more controlled) or also include multi-modal ICL experiments?*

> **IMPORTANT**
> ***Q3: Compute Budget** -- Some experiments (especially SAE-based FRE analysis and many-shot scaling) are compute-intensive. What is your available compute budget? (single GPU, multi-GPU, cloud?)*

> **WARNING**
> ***Q4: Timeline** -- Phase 1 is designed to take 4-8 weeks. Is this aligned with your research timeline, or do we need to prioritize a subset of experiments?*

Phase 1: Systematic Experimentation Protocol

Overview of Experimental Modules

```
graph TD
    A["Phase 1: Problem Identification"] --> B["Module 1: Latent Noise Injection"]
    A --> C["Module 2: Attention Distribution Skewing"]
    A --> D["Module 3: Label Space Flipping"]
    A --> E["Module 4: Feature Relevance Estimation"]
    A --> F["Module 5: Negative Demonstration (?-term)"]
    A --> G["Module 6: Task Vector Geometry"]

    B --> H["Cross-Module Analysis"]
    C --> H
    D --> H
    E --> H
    F --> H
    G --> H

    H --> I["Problem Formulation"]
    I --> J["Phase 2: Solution Design"]
```

Module 1: Latent Noise Injection (Continuous Perturbation)

From your research log -- probing how noise in value vectors affects the query vector update.

Hypothesis

ICL is fragile to perturbations in specific latent dimensions, and the fragility pattern reveals which dimensions the model relies on (implicit FRE). If ICL performs good FRE, perturbations in "irrelevant" dimensions should have minimal impact.

Experimental Design

```
# Pseudocode for Latent Noise Injection
for model in [GPT2, LLaMA2-7B, Pythia-1.4B]:
    for task in [SST-2, AGNews, TREC, DBPedia]:
        for layer_l in model.layers:
            for noise_scale in [0.01, 0.05, 0.1, 0.5, 1.0]:
                # Get clean activations
                h_clean = model.forward(context + query, return_hidden=True)
                V_clean = W_v @ h_clean[layer_l]  # Value vectors

                # Inject noise
                noise = torch.randn_like(V_clean) * noise_scale
                V_perturbed = V_clean + noise

                # Get perturbed output
                h_perturbed = model.forward_with_intervention(
                    context + query,
                    layer=layer_l,
                    new_values=V_perturbed
                )

                # Measure degradation
                cos_sim = cosine_similarity(h_clean[-1], h_perturbed[-1])
                acc_drop = accuracy(h_clean) - accuracy(h_perturbed)
                kl_div = KL(logits_clean, logits_perturbed)
```

Metrics

Metric	What It Measures
Cosine similarity decay curve	How quickly the query representation diverges
Accuracy degradation rate	Task performance sensitivity per layer
KL divergence of output logits	Distribution shift in predictions
Per-dimension sensitivity map	Which latent axes matter most (-> FRE)

Expected Outcomes

- **If ICL has implicit FRE**: Noise in task-irrelevant dimensions causes minimal degradation; noise in task-relevant dimensions causes catastrophic failure -> Clean separation pattern
- **If ICL lacks implicit FRE**: Degradation is uniform across dimensions -> All dimensions are equally weighted -> FRE is missing and can be added

Module 2: Attention Distribution Skewing (Structural Perturbation)

From your research log -- quantifying recency bias through demonstration reordering.

Hypothesis

The attention distribution over demonstrations reflects the QVM weighting scheme. If attention is dominated by recency bias rather than semantic relevance, then ICL's QVM is suboptimal -- it uses position-based weighting (β proportional to position) rather than content-based weighting.

Experimental Design

```
# Attention analysis for demonstration ordering effects
for model in models:
    for task in tasks:
        for k in [4, 8, 16, 32]:  # Number of demonstrations
            demos = select_demonstrations(task, k)

            for permutation_seed in range(100):  # 100 random orderings
                perm_demos = permute(demos, seed=permutation_seed)

                # Record
                output = model(perm_demos + query)
                attn_weights = model.get_attention_weights()
                query_vec = model.get_query_representation()

                # Analyze
                record(
                    ordering=permutation_seed,
                    prediction=output,
                    attention_entropy=entropy(attn_weights),
                    query_vector=query_vec,
                    accuracy=is_correct(output)
                )

            # Compute variance of query vectors across orderings
            query_variance = variance_across_permutations(query_vectors)
            prediction_stability = agreement_across_permutations(predictions)
```

Key Analyses

1. Query Vector Variance Map: Visualize in 2D (t-SNE/UMAP) how the final query vector moves under different orderings -> If high variance, QVM is order-dependent (a problem)

2. Attention Entropy vs. Position: Plot attention weights by position -> If monotonically increasing toward recent positions, recency bias dominates

3. Accuracy vs. Ordering: Correlate performance with ordering patterns -> Identify "best" and "worst" orderings

Connection to QVM Theory

In Rocchio QVM, the weight $\beta/|D^+|$ is uniform across all positive demonstrations. In ICL, softmax attention replaces this uniform weighting. The question is: **does softmax attention implement a better weighting than uniform, or does it introduce pathological biases (recency, primacy)?**

Module 3: Label Space Flipping (Semantic Perturbation)

From your research log -- observing whether ICL follows format or semantics.

Hypothesis

When labels are flipped, the query vector should move toward the "format manifold" (the syntactic pattern) rather than the "semantic manifold" (the correct answer). This reveals the relative strength of format learning vs. task learning in ICL's latent space manipulation.

Experimental Design

```
# Label flipping experiment
for model in models:
    for task in classification_tasks:
        for flip_ratio in [0.0, 0.25, 0.5, 0.75, 1.0]:
            demos = get_demonstrations(task, k=8)
            flipped_demos = flip_labels(demos, ratio=flip_ratio)

            for layer in model.layers:
                h_q = model.get_hidden_state(
                    flipped_demos + query,
                    layer=layer,
                    position='query'
                )

                # Project to 2D for visualization
                record_trajectory(layer, flip_ratio, h_q)

            # Record final prediction
            prediction = model.predict(flipped_demos + query)
            follows_flipped = (prediction == flipped_label)
            follows_correct = (prediction == correct_label)

            record(flip_ratio, follows_flipped, follows_correct)
```

Key Analyses

1. Format vs. Semantic Manifold: At what flip ratio does the model switch from following the correct answer to following the flipped format?

2. Layer-wise Trajectory: In which layers does the switch happen? Early layers may encode format, later layers encode semantics.

3. The "Crossover Point": The flip ratio at which the model transitions reveals the relative strength of format learning vs. semantic understanding -- this is a **quantifiable failure point**.

Connection to QVM/FRE

- In QVM terms: flipping labels changes D^+ to D^- -> the model should be pulled in the wrong direction
- In FRE terms: if the model has good FRE, it should weight semantic features (which remain correct) over label features (which are flipped)
- **If FRE is weak**: The model will follow flipped labels even when semantic features disagree -> This is the problem we want to expose

Module 4: Feature Relevance Estimation Probing (NEW -- FRE Analysis)

> **NOTE**
> *This is the **new contribution** integrating FRE from the TPAMI 2005 paper into ICL analysis.*

Hypothesis

Different latent dimensions (axes) of the hidden representation encode different features, and ICL implicitly weights these dimensions. By measuring which dimensions are most important for task performance, we can construct an explicit FRE map and compare it to the model's implicit FRE.

4a: Per-Dimension Sensitivity Analysis (Direct FRE Measurement)

```
# Measure per-dimension importance via ablation
for model in models:
    for task in tasks:
        d = model.hidden_dim  # e.g., 768 for GPT-2

        # Baseline accuracy with full ICL
        baseline_acc = evaluate_icl(model, task, k=8)

        # Ablate each dimension
        dimension_importance = torch.zeros(d)
        for dim_j in range(d):
            # Zero out dimension j at the query position
            h_ablated = ablate_dimension(model, task, dim=dim_j)
            acc_ablated = evaluate_icl_with_intervention(model, task, h_ablated)
            dimension_importance[dim_j] = baseline_acc - acc_ablated

        # This gives us the "ground truth" FRE weights
        fre_weights = dimension_importance / dimension_importance.sum()
```

4b: Attention Head Specialization (Implicit FRE Discovery)

```
# Analyze which attention heads focus on which feature dimensions
for model in models:
    for task in tasks:
        for head_h in model.attention_heads:
            # Get the output of this head for the query token
            head_output = model.get_head_output(head_h, context + query)

            # Measure which dimensions this head affects most
            head_contribution = head_output.abs().mean(dim=0)  # [d]

            # Correlate with FRE weights from 4a
            correlation = pearsonr(head_contribution, fre_weights)
            record(head_h, correlation, head_contribution)
```

4c: SAE-Based Feature Decomposition (Explicit FRE)

```
# Use Sparse Autoencoders to decompose activations into interpretable features
# Then measure which SAE features are relevant for ICL

# Load pre-trained SAE (e.g., from Anthropic or EleutherAI)
sae = load_sae(model_name, layer=target_layer)

for task in tasks:
    for example in task.test_set:
        # Get activations during ICL
        h_icl = model.get_hidden(context + query, layer=target_layer)

        # Decompose into SAE features
        features = sae.encode(h_icl)  # Sparse vector of feature activations

        # Identify which SAE features change most between
        # zero-shot and ICL conditions
        h_zeroshot = model.get_hidden(query_only, layer=target_layer)
        features_zeroshot = sae.encode(h_zeroshot)

        # The difference reveals ICL's implicit FRE
        icl_fre_map = features - features_zeroshot
```

Metrics

Metric	What It Measures
Dimension importance vector	Ground-truth FRE via ablation
Head-FRE correlation	How well attention heads implement FRE
SAE feature activation delta	Which interpretable features ICL activates
FRE entropy	How concentrated or distributed the feature weighting is
Cross-task FRE overlap	Whether the same dimensions matter across tasks

Expected Outcomes & Problem Identification

- **Outcome A**: ICL has strong implicit FRE -> Dimension importance is sharply concentrated on task-relevant axes -> The model knows what to pay attention to
- **Outcome B**: ICL has weak/noisy FRE -> Dimension importance is diffuse -> The model wastes capacity on irrelevant features -> **PROBLEM: ICL lacks effective feature selection, and explicit FRE injection could improve it**
- **Outcome C**: FRE varies wildly across demonstrations -> The model's feature weighting is unstable -> **PROBLEM: ICL's implicit FRE is not robust**

Module 5: Negative Demonstration Experiments (The ?-Term Gap)

From your research log -- the "missing $-\gamma$ term" identified as a research gap.

Hypothesis

Standard ICL lacks a repulsion mechanism. By engineering negative demonstrations and measuring their effect on the query vector trajectory, we can quantify the value of the missing ?-term and design a mechanism to implement it.

5a: Contrastive ICL Experiment

```
# Compare standard ICL vs. ICL with negative demonstrations
for model in models:
    for task in tasks:
        # Standard ICL: only positive demos
        pos_demos = get_positive_demos(task, k=4)
        acc_standard = evaluate(model, pos_demos + query)
        h_standard = model.get_query_vector(pos_demos + query)

        # Contrastive ICL: positive + negative demos
        neg_demos = get_negative_demos(task, k=4)  # Wrong labels
        contrastive_prompt = format_contrastive(pos_demos, neg_demos, query)
        acc_contrastive = evaluate(model, contrastive_prompt)
        h_contrastive = model.get_query_vector(contrastive_prompt)

        # Analyze the difference
        delta_h = h_contrastive - h_standard  # The "?-term effect"

        # Measure if the query vector moved AWAY from negative examples
        for neg in neg_demos:
            h_neg = model.encode(neg)
            repulsion = cosine_similarity(h_contrastive, h_neg)
            attraction = cosine_similarity(h_standard, h_neg)
            record(repulsion_change = attraction - repulsion)
```

5b: Synthetic ?-Term Injection

```
# Directly implement the Rocchio ?-term in latent space
for model in models:
    for task in tasks:
        h_q_standard = model.get_query_vector(pos_demos + query)

        # Compute the negative centroid
        neg_centroid = mean([model.encode(neg) for neg in neg_demos])

        # Apply Rocchio ?-term
        for gamma in [0.1, 0.3, 0.5, 0.7, 1.0]:
            h_q_modified = h_q_standard - gamma * neg_centroid

            # Re-inject and evaluate
            acc_modified = evaluate_with_modified_query(model, h_q_modified)
            record(gamma, acc_modified)
```

Expected Outcomes

- **If ?-term helps**: Accuracy improves when repulsion is added -> **PROBLEM: ICL lacks negative feedback, and implementing it yields gains**
- **If ?-term hurts at high ?**: There's an optimal ? that depends on the task -> **PROBLEM: The repulsion strength needs to be learned, not fixed**
- **If models already learn implicit repulsion**: Attention weights on negative demos are already negative-like -> The gap may be smaller than theorized

Module 6: Task Vector Geometry Analysis

Building on Hendel et al. (2023) "In-Context Learning Creates Task Vectors" and Todd et al. (ICLR 2024) "Function Vectors."

Hypothesis

The "task vector" created by ICL demonstrations occupies a specific region of latent space. By analyzing its geometry, we can understand the structure of the QVM movement and identify when it fails.

6a: Task Vector Extraction & Comparison

```
# Extract task vectors for different tasks
for model in models:
    task_vectors = {}
    for task in tasks:
        # Task vector = h_ICL - h_zeroshot (at query position)
        h_icl = model.get_hidden(demos + query, layer=L, position='query')
        h_zero = model.get_hidden(query, layer=L, position='query')
        task_vectors[task] = h_icl - h_zero

    # Analyze geometry
    # 1. Are task vectors separable?
    for t1, t2 in pairs(tasks):
        similarity = cosine_similarity(task_vectors[t1], task_vectors[t2])
        record(t1, t2, similarity)

    # 2. Do task vectors cluster by task type?
    visualize_tsne(task_vectors)

    # 3. Is the task vector stable across different demo sets?
    for task in tasks:
        vectors_across_demos = []
        for demo_set in sample_demo_sets(task, n=50):
            tv = extract_task_vector(model, demo_set, query)
            vectors_across_demos.append(tv)
        stability = variance(vectors_across_demos)
        record(task, stability)
```

6b: Task Vector Arithmetic (QVM Superposition)

```
# Can we combine task vectors like QVM combines feedback?
# Task arithmetic: tv(A) + tv(B) should yield a model that does both A and B

tv_sentiment = extract_task_vector(model, sentiment_demos)
tv_topic = extract_task_vector(model, topic_demos)

# Combined task
tv_combined = tv_sentiment + tv_topic
acc_combined = evaluate_with_task_vector(model, tv_combined, combined_test)
```

Cross-Module Analysis: Convergence to Problem Formulation

After running all modules, perform the following meta-analyses:

1. Failure Mode Taxonomy

```
graph LR
    A["Module Results"] --> B{"Failure Classification"}
    B --> C["Structural Failures"]
    B --> D["Semantic Failures"]
    B --> E["Feature Selection Failures"]
    B --> F["Scaling Failures"]

    C --> G["Recency bias dominates<br/>attention weighting"]
    D --> H["Format learning overpowers<br/>semantic understanding"]
    E --> I["Implicit FRE is weak or<br/>unstable across tasks"]
    F --> J["Performance degrades<br/>at many-shot scale"]
```

2. Unified Sensitivity Matrix

Construct a matrix: **Task × Perturbation Type × Layer -> Degradation Score**

This matrix reveals:

- Which tasks are most vulnerable
- Which perturbation types expose the most failures
- Which layers are critical for ICL
- Where FRE is strongest/weakest

3. Problem Formulation Decision Tree

```
IF Module 4 shows weak implicit FRE:
    PROBLEM: "ICL lacks explicit feature relevance estimation,
    leading to suboptimal latent space navigation"
    SOLUTION DIRECTION: Inject explicit FRE via learned axis weighting

IF Module 5 shows ?-term helps significantly:
    PROBLEM: "ICL operates with only positive feedback,
    missing the repulsive component of classical relevance feedback"
    SOLUTION DIRECTION: Design contrastive attention or repulsive
    residual connections

IF Module 2 shows severe recency bias:
    PROBLEM: "ICL's QVM weighting is corrupted by positional bias,
    deviating from content-based relevance weighting"
    SOLUTION DIRECTION: Positional debiasing of attention or
    Rocchio-inspired uniform weighting

IF Module 3 shows early format dominance:
    PROBLEM: "ICL encodes format before semantics, creating a
    vulnerability where wrong labels in correct format override
    correct semantic understanding"
    SOLUTION DIRECTION: Layer-specific interventions to prioritize
    semantic features

IF Module 6 shows unstable task vectors:
    PROBLEM: "Task vectors are not robust to demonstration selection,
    making ICL unreliable for practical deployment"
    SOLUTION DIRECTION: Task vector regularization or demonstration-
    agnostic task encoding
```

Evaluation Framework

Benchmarks (Ordered by Complexity)

Level	Dataset	Task	# Classes	Why
Easy	SST-2	Sentiment	2	Baseline binary classification
Easy	MR	Sentiment	2	Domain variation from SST-2
Medium	AGNews	Topic	4	Multi-class classification
Medium	TREC	Question Type	6	Fine-grained classification
Hard	DBPedia	Ontology	14	Many-class, complex
Hard	SST-5	Fine-grained Sentiment	5	Subtle distinctions
Reasoning	GSM8K (subset)	Math	N/A	Tests beyond pattern matching
Reasoning	BoolQ	QA	2	Tests reading comprehension

Models (Ordered by Size)

Model	Parameters	Why
GPT-2 Small	124M	Fast iteration, full interpretability
GPT-2 Large	774M	Scale comparison within same family
Pythia-1.4B	1.4B	Strong mechanistic interpretability support
Pythia-6.9B	6.9B	Scaling behavior
LLaMA-2-7B	7B	Production-relevant scale
LLaMA-3-8B	8B	Latest architecture

Metrics Summary

Category	Metric	Modules
Performance	Accuracy, Macro-F1	All
Calibration	Expected Calibration Error (ECE)	1, 3, 5
Representation	Cosine similarity, L2 distance	1, 2, 6
Distribution	KL divergence of logits	1, 3
Stability	Variance across perturbations	2, 6
Feature Importance	Dimension ablation score	4
Geometry	Task vector inter/intra-class distance	6
FRE-specific	FRE entropy, cross-task FRE overlap	4
ICL-specific	Learning-to-Context Slope (LCS)	All

What NOT To Do

> **CAUTION**
> ***Anti-Patterns to Avoid in Phase 1***

1. Don't jump to solutions -- Phase 1 is about diagnosis, not treatment. Resist the urge to propose architectural changes before you have data showing where ICL actually fails.

2. Don't overfit to one model -- Results on GPT-2 alone are insufficient. Cross-model validation is essential to ensure findings are about ICL (the phenomenon) not a specific architecture.

3. Don't conflate prompt engineering with ICL theory -- We are studying the *mechanism*, not optimizing prompts. Keep prompts standardized (StaICC-style templates) to isolate the variable of interest.

4. Don't ignore null results -- If ICL turns out to be robust to a perturbation, that's a finding. It means the model has learned something interesting about that dimension. Report it.

5. Don't run all experiments at once -- Follow the priority order. Module 4 (FRE) and Module 5 (?-term) are most novel and should be prioritized if compute is limited.

6. Don't use only black-box metrics -- Accuracy alone is insufficient. The latent space analysis (cosine similarity, task vector geometry, FRE maps) is where the novel contributions live.

7. Don't skip statistical rigor -- All experiments should use multiple random seeds (>=5), report confidence intervals, and use appropriate statistical tests (paired t-tests, bootstrap).

8. Don't ignore existing baselines -- Compare against supervised probing baselines, fine-tuning results, and existing ICL optimization methods (e.g., KATE, EPR for demonstration selection).

Timeline & Priority

Priority Order (If Compute-Constrained)

Priority	Module	Estimated Time	Novelty
[P0] P0	Module 4: FRE Probing	2 weeks	*** Highest -- Novel connection
[P0] P0	Module 5: ?-Term Experiments	1.5 weeks	*** High -- Direct from your theory
[P1] P1	Module 3: Label Space Flipping	1 week	** Extends Min et al. 2022
[P1] P1	Module 6: Task Vector Geometry	1.5 weeks	** Extends Hendel et al. 2023
[P2] P2	Module 1: Latent Noise Injection	1 week	* Establishes baselines
[P2] P2	Module 2: Attention Skewing	1 week	* Well-studied, supports claims

Week-by-Week Plan

Week	Activity
Week 1	Set up experimental infrastructure: model loading, activation extraction hooks, evaluation pipeline
Week 2	Run Module 4a (Per-Dimension Sensitivity) on GPT-2 + SST-2/AGNews
Week 3	Run Module 4b-c (Attention Head FRE, SAE decomposition) + Module 5a (Contrastive ICL)
Week 4	Run Module 5b (?-term injection) + Module 3 (Label Flipping)
Week 5	Run Module 6 (Task Vector Geometry) + Module 1 (Noise Injection)
Week 6	Run Module 2 (Attention Skewing) + Scale up key findings to larger models
Week 7	Cross-module analysis, unified sensitivity matrix, failure taxonomy
Week 8	Problem formulation, writeup, Phase 2 planning

Verification Plan

Automated Tests

- Unit tests for all activation extraction and intervention utilities
- Reproducibility tests: same seed -> same results
- Sanity checks: zero noise -> baseline accuracy, full label flip -> chance accuracy

Statistical Validation

- All results reported with mean +/- std over >=5 seeds
- Paired t-tests for comparing conditions
- Bootstrap confidence intervals for accuracy metrics
- Effect sizes (Cohen's d) for all comparisons

Manual Verification

- Visual inspection of t-SNE/UMAP plots for task vector geometry
- Qualitative analysis of examples where ICL fails under perturbation
- Comparison of FRE maps with human intuition about task-relevant features

Key Literature Reference Table

Paper	Year	Relevance
Yin, Bhanu, Chang, Dong -- TPAMI	2005	FRE + QVM + BI framework
Brown et al. -- GPT-3	2020	ICL foundation
Min et al. -- Rethinking Demonstrations	2022	Format vs. semantics in ICL
Dong et al. -- ICL Survey	2022	Comprehensive ICL landscape
Kossen et al. -- Label Relationships	2023	Counter to Min et al.
von Oswald et al. -- Transformers as GD	2023	ICL as implicit optimization
Dai et al. -- Why Can GPT Learn ICL?	2023	Dual attention-gradient view
Akyürek et al. -- Linear Models	2023	ICL as regression
Hendel et al. -- Task Vectors	2023	Task vector extraction
Todd et al. -- Function Vectors	2024	Function vector discovery
Li et al. -- In-context Vectors	2024	Latent space steering
Anthropic -- Scaling Monosemanticity	2024	SAE for interpretable features
Contrastive ICL papers	2024-25	Negative demonstration methods

Proposed Deliverables (Phase 1)

1. Codebase: Modular experimental framework with clean APIs for activation extraction, intervention, and evaluation

2. FRE Analysis Report: First systematic study connecting classical FRE to ICL attention dynamics

3. Failure Taxonomy: Categorized failure modes of ICL viewed as latent space manipulation

4. Problem Statement: A precise, well-motivated research problem with quantitative evidence

5. Phase 2 Proposal: Solution direction with preliminary evidence from Phase 1

This plan will be updated as experimental results come in. Phase 2 (Solution Design) will be initiated once a clear problem formulation emerges from Phase 1 diagnostics.